

Roadmap: Panel Administrativo Django

Roles: Super Admin (crea/gestiona Admins y Usuarios, control total), Admin (gestiona Usuarios dentro de su alcance), Usuario (acceso a su propio panel).

Hito 1: Arquitectura base y configuracion

- Crear proyecto Django con apps: core, accounts, dashboard, panel_admin.
- Configurar entorno virtual y variables de entorno (.env) con django-environ para SECRET_KEY, DEBUG y credenciales de BD.
- Definir base de datos (PostgreSQL en produccion; SQLite solo en desarrollo).
- Configurar settings separados: settings/base.py, dev.py, prod.py.

Hito 2: Modelo de usuarios y roles jerarquicos

- Crear modelo de usuario personalizado (AbstractUser) desde el inicio, con campo role o usando Groups y Permissions nativos de Django.
- Definir jerarquia: Super Admin gestiona Admins y Usuarios; Admin solo gestiona Usuarios de su alcance; Usuario ve solo su panel.
- Usar django.contrib.auth Groups/Permission, o django-guardian para permisos a nivel de objeto.
- Crear el Super Admin inicial via management command (manage.py crear_superuser) en lugar de exponerlo en la interfaz.
- Restringir vistas por rol con LoginRequiredMixin y UserPassesTestMixin o decoradores personalizados.

Hito 3: CRUD de administracion de usuarios

- Vista para que Super Admin cree, edite, desactive y elimine cuentas de Admin.
- Vista para que Admin cree y gestione cuentas de Usuario, sin acceso a otros Admins.
- Formularios con validacion robusta (ModelForms), sin exponer campos sensibles como is_superuser.
- Paginacion y filtros de busqueda en listados de usuarios (nombre, rol, estado).
- Auditoria de cambios (quien creo/modifico/elimino y cuando) con django-simple-history o modelo de logs propio.

Hito 4: Dashboard simple por rol

- Metricas basicas: total de usuarios, admins activos, ultimos registros, actividad reciente.
- Vistas diferenciadas por rol: Super Admin ve datos globales, Admin ve solo su equipo, Usuario ve su perfil.
- Graficos simples con Chart.js integrados via templates, evitando dependencias pesadas.
- Template base responsivo (Bootstrap 5 o Tailwind) con sidebar de navegacion condicional segun permisos.

Hito 5: Seguridad

- Forzar HTTPS en produccion (SECURE_SSL_REDIRECT, SESSION_COOKIE_SECURE, CSRF_COOKIE_SECURE).
- Proteccion CSRF activa en todos los formularios y validacion de tokens en peticiones AJAX.
- Limitar intentos de login con django-axes o django-ratelimit contra fuerza bruta.
- Politicas de contraseñas robustas (AUTH_PASSWORD_VALIDATORS) y hashing con Argon2.
- Bloquear escalamiento de privilegios: un Admin nunca debe poder autoasignarse rol de Super Admin (validar en backend).
- 2FA opcional con django-two-factor-auth para cuentas Super Admin y Admin.
- Configurar ALLOWED_HOSTS y CORS restringido solo a dominios necesarios.
- Usar el ORM de Django (evita SQL injection) y sanitizar entradas para prevenir XSS.
- Registrar logs de seguridad: intentos fallidos de login y cambios de permisos.

Roadmap: Panel Administrativo Django

Hito 6: Pruebas, despliegue y lineamientos

- Tests unitarios y de integracion por rol con pytest-django, verificando accesos permitidos y denegados.
- Ejecutar "python manage.py check --deploy" antes de produccion.
- Backups automaticos de base de datos y variables de entorno fuera del control de versiones (.gitignore con .env).
- Documentar roles, permisos y flujo de creacion de cuentas para mantenimiento futuro.
- Despliegue con Gunicorn + Nginx, y Whitenoise o CDN para archivos estaticos.

Lineamientos generales

- Principio de menor privilegio: cada rol solo con los permisos estrictamente necesarios.
- Centralizar logica de permisos en un modulo reutilizable (permissions.py).
- Versionar el esquema de roles y permisos mediante migraciones controladas.
- Revisar periodicamente accesos y desactivar cuentas inactivas.